

1: Transformer LM resource accounting

Part (a)

HYPERPARAMETER (GPT2-XL)	VALUE
VOCAB_SIZE	50,257
CONTEXT_LENGTH	1024
NUM_LAYERS	48
NUM_HEADS	25
D_FF	6,400
D_MODEL	1,600

S. No.	Layer	Size	Value
1	Token embeddings	VOCAB_SIZE x D_MODEL	50,257 x 1,600 = 80,411,200
2	Transformer Block (RMSNorm) x 2	D_MODEL x 2	1,600 x 2 = 3,200
3	Transformer Block (Causal Multihead Attention)	(Q,K,V,O): D_MODEL x D_MODEL x 4	1,600 x 1,600 x 4 = 10,240,000
4	Transformer Block (SwiGLU)	(W1, W3): D_MODEL x D_FF x 2 (W2) : D_FF x D_MODEL	1,600 x 6,400 x 3 = 30,720,000
5	Final RMSNorm	D_MODEL	1,600
6	Final Linear	D_MODEL x VOCAB_SIZE	1,600 x 50,257 = 80,411,200

Total trainable parameters:

$$\begin{aligned}
 &= \#(\text{Token embeddings}) + 48 * (\#(\text{Transformer Block})) + \#(\text{Final RMSNorm}) + \#(\text{Final Linear}) \\
 &= 80,411,200 + 48 * (3,200 + 10,240,000 + 30,720,000) + 1,600 + 80,411,200 \\
 &= 2,127,057,600 \text{ (~2.13 B)}
 \end{aligned}$$

Total memory required: Number of trainable parameters x 4 (1 FLOP = 4 Bytes)

$$= 8,508,230,400 \text{ Bytes} = 7.924 \text{ GB}$$

Part (b)

MatMul ($A[m,n]$, $B[n,p]$) -> $2mnp$ FLOPS, C: context length, D_m: D_MODEL

1	Attention Block	Matrix Multiplications Involved	FLOPS calculation	Values	% FLOPS
2		W.Q, W.K, W.V	$3 \times [2 \times C \times D_m \times D_m]$	$3 \times [2 \times 1,024 \times 1,600 \times 1,600] = 15,728,640,000$	6.16%
3		Q.transpose(K)	$(C, D_K) \times (D_K, C) \times \text{Num_heads} = 2 \times C^2 \times D_K \times \text{Num_heads} = 2 \times C^2 \times D_m$	$2 \times 1,024^2 \times 1,600 = 3,355,443,200$	1.31%
4		Attention.V	$(C, C) \times (C, D_m) = 2 \times C^2 \times D_m$	$2 \times 1,024^2 \times 1,600 = 3,355,443,200$	1.31%

5		Output Projection (O)	$(C, D_m) \times (D_m, D_m)$ $= 2 \times c \times D_m^2$	$2 \times 1,024 \times 1,600^2 = 5,242,880,000$	2.05%
6		SwiGLU	$(W1, W3): (C, D_m) \times (D_m, D_{FF})$ $(W2): (C, D_{FF}) \times (D_{FF}, D_m)$ $= 3 \times [2 \times C \times D_m \times D_{FF}]$	$3 \times 2 \times 1,024 \times 1,600 \times 6,400 = 62,914,560,000$	24.65%
7	Final Linear	$(C, D_m) \times (D_m, \text{VOCAB_SIZE})$	$2 \times C \times D_m \times \text{VOCAB_SIZE}$	$2 \times 1,024 \times 1,600 \times 50,257 = 164,682,137,600$	64.51%

Total Flops:

$$= 48 \times (15,728,640,000 + 3,355,443,200 + 3,355,443,200 + 5,242,880,000 + 62,914,560,000) + 164,682,137,600$$

$$= 4,513,336,524,800$$

Part (c)

The final linear layer requires the highest number of FLOPS which is ~65% in this example. It is because the final multiplication depends on the VOCAB_SIZE which is very large compared to D_MODEL, CONTEXT_LENGTH and D_FF

Part (d)

Assuming the context length and vocab size remains same and D_FF = 4 D_Model for all GPT family models

	GPT-2 small	GPT-2 medium	GPT-2 large
Context_Length (C)	1,024	1,024	1,024
Vocab_size	50,257	50,257	50,257
D_model	768	1024	1280
D_ff	3072	4096	5120
Num_Heads	12	16	20
Num_layers	12	24	36

Blocks	Operations	GPT-2 small	% Flops	GPT-2 medium	% Flops	GPT-2 large	% Flops
Attention Block	W.Q, W.K, W.V	3,623,878,656	2.92%	6,442,450,944	3.53%	10,066,329,600	4.03%
	Q.transpose(K)	1,610,612,736	1.30%	2,147,483,648	1.18%	2,684,354,560	1.07%
	Attention.V	1,610,612,736	1.30%	2,147,483,648	1.18%	2,684,354,560	1.07%
	Output Projection (O)	1,207,959,552	0.97%	2,147,483,648	1.18%	3,355,443,200	1.34%
	SwiGLU	14,495,514,624	11.68%	25,769,803,776	14.10%	40,265,318,400	16.12%
Total Attention		22,548,578,304	18.16%	38,654,705,664	21.16%	59,055,800,320	23.64%
Final Linear		79,047,426,048	63.67%	105,396,568,064	57.69%	131,745,710,080	52.73%
Total		349,630,365,696		1,033,109,504,000		2,257,754,521,600	

As the model size increases while context length and vocab size remain fixed. SwiGLU takes up the most FLOPs. The OKV and output projections also increase because they scale quadratically with d_model however the %increase is small. Q.transpose(K) and Attention.V decreases as they scale linearly with d_model. Final linear FLOPs share decreases sharply because it only scales once with d_model whereas the attention block operations scale with the number of layers as well.

Part (e)

Blocks	Operations	GPT-2 XL	% Flops	GPT-2 XL-2	% Flops
Attention Block	W.Q, W.K, W.V	15,728,640,000	4.55%	251,658,240,000	2.87%
	Q.transpose(K)	3,355,443,200	0.97%	858,993,459,200	9.81%
	Attention.V	3,355,443,200	0.97%	858,993,459,200	9.81%
	Output Projection (O)	5,242,880,000	1.52%	83,886,080,000	0.96%
	SwiGLU	62,914,560,000	18.19%	1,006,632,960,000	11.50%
Total Attention		90,596,966,400	26.19%	3,060,164,198,400	34.95%
Final Linear		164,682,137,600	47.61%	2,634,914,201,600	30.10%
Total		4,513,336,524,800		149,522,795,724,800	

When we increase the context length the Q.transpose(K) and Attention.V increases quadratically. Remaining components scale linearly due to which the %FLOPs reduces especially for SwiGLU.

2: AdamW accounting

Part (a)

Let vocab_size: V , context_length: C , num_layers: L , d_model: d , num_heads: h , batch_size: B and $d_{ff} = 4d$.

Number of parameters:

1. Token Embedding = Vd
2. Per Transformer Block:
 - a. Q, K, V and Output projections: $(d, d) = 4d^2$
 - b. RMSNorm (2 times inside the block) = $2d$
 - c. SwiGLU($W1, W2, W3$) (d, d_{ff}) = $3 \cdot d \cdot d_{ff} = 12d^2$
3. Final Linear Layer = Vd
4. Final RMSNorm = d
- Total params (P) = $2Vd + d + L \cdot (2d + 16d^2)$
 - Total memory required = $4P$
 - There will be 1 gradient per parameter so same space = $4P$
 - There are 2 moments per parameter so total = $8P$
 - Activations:
 - Per transformer block:
 - RMSNorm (total 2) = $2 \cdot B \cdot C \cdot d$
 - Q, K, V projections = $3 \cdot B \cdot C \cdot d$
 - Q.transpose(K) = $B \cdot h \cdot C \cdot C$
 - Softmax = $B \cdot h \cdot C \cdot C$
 - Attention.V = $B \cdot C \cdot d$
 - Output projection = $B \cdot C \cdot d$
 - SwiGLU ($W1 + \text{SiLU} + W2$) = $B \cdot C \cdot d_{ff} + B \cdot C \cdot d_{ff} + B \cdot C \cdot d = 2 \cdot B \cdot C \cdot 4d + B \cdot C \cdot d = 9 \cdot B \cdot C \cdot d$
 - Total transformer for L blocks = $L \cdot (16 \cdot B \cdot C \cdot d + 2 \cdot B \cdot h \cdot C^2)$
 - Final RMSNorm = $B \cdot C \cdot d$
 - Final Linear = $B \cdot C \cdot V$
 - Hence total memory required for activations $A = L \cdot (16 \cdot B \cdot C \cdot d + 2 \cdot B \cdot h \cdot C^2) + B \cdot C \cdot d + B \cdot C \cdot V$

Total memory required

$$= 4P + 4P + 8P + 4A$$

$$= 16P + 4A$$

$$= 16(2Vd + d + L.(2d + 16d^2)) + 4(L.(16.B.C.d + 2.B.h.C^2) + B.C.d + B.C.V)$$

Part (b)

GPT-2 XL: $V = 50,257$, $C = 1,024$, $L = 48$, $d = 1,600$, $h = 25$, $d_{ff} = 6,400$, $batch_size = B$

Total memory required:

$$M = 16 * (2*50257*1600 + 1600 + 48*(2*1600 + 16*1600*1600)) + 4*(48*(16*B*1024*1600 + 2*B*25*1024*1024) + B*1024*1600 + B*1024*50257)$$

$$M = 34,032,921,600 + 15,517,753,344 \cdot B \text{ (bytes)}$$

Now

- $M \leq 80 \text{ GiB}$
- $34,032,921,600 + 15,517,753,344 \cdot B \leq 85,899,345,920$
- $B \leq (85,899,345,920 - 34,032,921,600)/15,517,753,344$
- $B \leq 3.3423$

Hence the **maximum batch_size (B) = 3**

Part (c)

FLOPs per AdamW operations (per parameter):

1. $m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g \rightarrow 2 \text{ Multiply and 1 add (3 operations)}$
2. $v \leftarrow \beta_2 \cdot v + (1 - \beta_2) \cdot g^2 \rightarrow 1 \text{ square, 2 Multiply and 1 add (4 operations)}$
3. $\theta \leftarrow \theta - \alpha_t \cdot m / (\sqrt{v} + \epsilon) \rightarrow 1 \text{ square root, 1 add, 1 divide, 1 subtract and 1 multiply (5 operations)}$
4. $\alpha_t \leftarrow \alpha \cdot \sqrt{(1 - \beta_2^t)/(1 - \beta_1^t)} \rightarrow \text{Scaler operation and happen once per step so negligible}$
5. $\theta \leftarrow \theta - \alpha \lambda \rightarrow 1 \text{ Multiply and 1 add (2 operations)}$

Adding all gives a total of 14 operations per parameter.

$$\text{Hence FLOPS} \approx 14P = 14 \cdot (2Vd + d + L.(2d + 16d^2))$$

Part (d)

Theoretical peak FLOPs of NVIDIA A100 = 19.5 TeraFLOP / s

Total FLOPs required = FLOPs forward + FLOPs backward = 3 x (FLOPs forward)

FLOPs forward:

- Per transformer block MatMuls (each mat-mul takes $2mnp$ for $A(m, n)$ and $B(n, p)$ matrices)
 - Q,K,V projections: $3 \cdot (B.C.d)@(d, d) = 6B.C.d^2$
 - Q.transpose(K): $(B, h, C, d_k)@(B, h, d_k, C) = \sum_{B,h} 2 \cdot d_k \cdot C^2 = 2 \cdot B \cdot d \cdot C^2$
 - Attention.V: $2 \cdot B \cdot d \cdot C^2$
 - Output projection: $(B, C, d)@(d, d) = 2B.C.d^2$
 - SwiGLU: $(W_1, W_2, W_3) \rightarrow (B, C, d)@(d, d_{ff} = 4d) = 3 \cdot 2 \cdot B \cdot C \cdot 4d^2 = 24B.C.d^2$
- Total transformer: $36B.C.d^2.L$
- Final Linear: $(B.C.d)@(d, V) = 2BCdV$
- Total Forward FLOPs: $36B.C.d^2.L + 2B.C.d.V$

For C = 1024, d = 1600, d_ff = 6400, V = 50257, L = 48, B = 1024

Total FLOPS: $3 * (36 * 1024 * 1024 * 1600^2 * 48 + 2 * 1024 * 1600 * 50257) = 13,916,188,085,452,800$

For 400k steps Total FLOPs: $400,000 * 13,916,188,085,452,800 = 5.57 \times 10^{21}$

Total days required to train on 50% MFU = $5.57 \times 10^{21} / (0.5 * 19.5 \times 10^{12}) = 57,12,82,051 \text{ seconds} = 6,612 \text{ days}$

3: Learning rate tuning

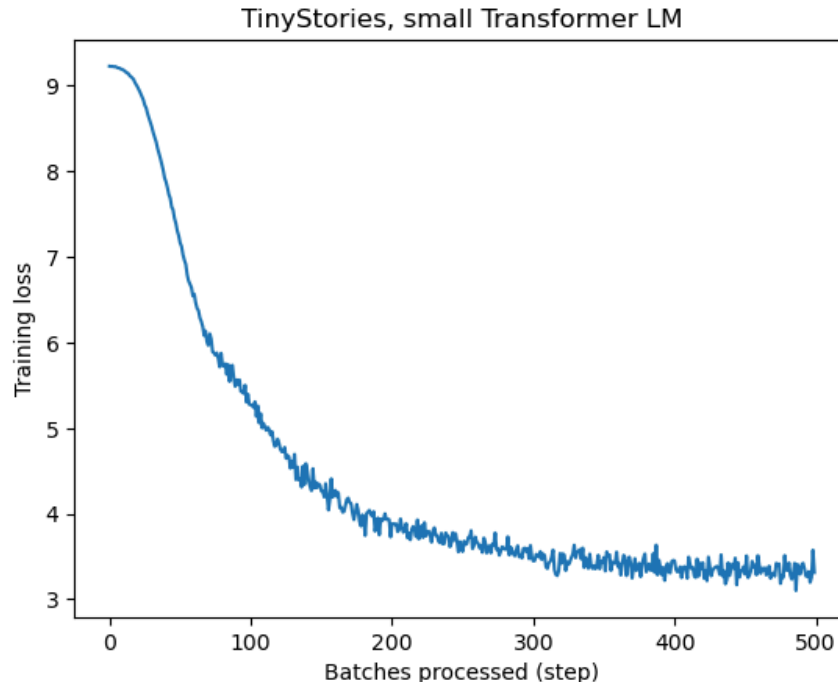
```
lr=10
losses=[26.3169, 16.8428, 10.7794, 6.8988, 4.4152, 2.8258, 1.8085, 1.1574, 0.7408, 0.4741]
lr=100
losses=[24.7994, 24.7994, 24.7994, 24.7994, 24.7994, 24.7994, 24.7994, 24.7994, 24.7994,
24.7994]
lr=1000
losses=[24.0794, 8692.668, 3138052.75, 1132836992.0, 408954109952.0, 147632437592064.0,
5.329530857822618e+16, 1.9239606306545336e+19, 6.94549750722696e+21, 2.5073241125942727e+24]
```

For lr = 10 loss is decreasing and it eventually starts to converge

For lr = 100 loss is constant (very minor change) hence no learning at all

For lr = 1000 loss exploded quickly and resulting in very unstable training

4: First training run



The model is trained for 500 steps. Training loss decreased overtime but it started fluctuating a lot depending on the batch after a few steps.

The actual text:

u don't have to be scared of the loud dog, I'll protect you". The mole felt so safe with the little girl. She was very kind and the mole soon came to trust her. He leaned against her and she kept him safe.

Initial decoding result:

u don't have to be scared of the loud liqu nibb novel Be winding quitecovered?" strings backpackTina
inter drink ends activity JamieHa scaring Jolly hundred appear keyhole bunny since connected rest
fleasractedFind chopping pearsitely glove FreddieSince bankarac attach cutting Grumpy lotion True gl
body spending deter afterwardsSince bank Since gigg folds Open disturbing Quacky dreaming choppingout

After training LM decoding result:

u don't have to be scared of the loud noise.

Once upon a time, there was a little girl named Lily. She loved to play with his friends. One day, he
saw a big, big tree. She was very happy. She wanted to play with the ball.
The little girl was very happy. She wanted to play

Observation from the decoding: Even after training the model has not done a good decoding work. However we
can see that before training it generated complete garbage whereas after training model is at least generating
words that make sense and are somewhat coherent.

5: Decoding

Decoding logic is implemented in the student/decoding.ipynb

PARAMS: Prompt length = 20, Max new tokens = 30, Temperature = 0.9, p = 0.7

Example output:

Prompt: very sad. The big lion wanted to help his friend. He roared very loud and asked all the

Generated: very sad. The big lion wanted to help his friend. He roared very loud and asked all the
pig and the best. But then, the cat went outside and his mom wanted to keep the tasty friend. They
went home and saw a big
